Also note that like many other built-in PHP array functions, `array_walk()` comes in a multi-dimensional flavor. If you need to apply your callback to elements in sub-arrays use `array_walk_recursive()`.

Creating New Arrays with `array_map()`

An alternative to `array_walk()` is `array_map()`, which also applies a callback function to every element in the array. The difference is that `array_map()` does not make any changes to the input array. Instead, it returns a new array, leaving the original data untouched. This has drawbacks in memory usage, but also (arguably) provides a benefit.

Anything (that is legal PHP code) can happen within `array_walk()` (just as it can in a `for` or `foreach` loop), which can introduce cognitive load. There is no guarantee that `array_walk()` – or a regular loop construct – won't trash your data. If the callback function is complex or large, this gets harder to see at a glance. `array_map()` guarantees that only new data will be introduced and for that reason some programmers prefer it:

```
$cars = [
    [
        ['make', 'VW'], ['model', 'Golf GTI']
    ],
    [
        ['make', 'VW'], ['model', 'Corrado VR6']
    ]
];
$values = [
    20000,
    12000
];
```